PAC-MAN CLONE

Project Report - Group 11

Overview

In this project, we developed a complete **Pac-Man game** clone written entirely in the **C language**. We have used the graphics library **Raylib** to create a **visual interface**, handle rendering, and manage user input. Our key functionalities include implementing **core mechanics** like ghost and Pac-Man movement, collision detection, and **storing players' data** like high scores, resulting in a functional 2D game.

This report will explain all the functionalities and the contribution of members in making them in detail.

Team Members :

- RAVI KARIYA – 202501112
- DESAI SANVI KALPIT – 202501113
- VADHVANA VISHESH JIGNESHKUMAR – 202501114
- PRANAV MADHAV BIRAJDAR – 202501115
- ARYAN GAMBHAVA – 202501116
- PIYUS PRATIK JENA – 202501117
- KATHAN JITENDRAKUMAR SAMDANI – 202501118
- RAVAL DHARMIK PRAVINBHAI – 202501119
- GOHIL ROHAN DIPAKBHAI – 202501120
- PADSUMBIYA DEV MAHESHBHAI – 202501121
- BHINDI MEET GHANSHYAMBHAI -- 202501122

Final Functionalities :

I. Core Game -

- UI & map development
- Movement of Ghost and Pac-Man

II. Other functionalities -

- File/User data management
- Music
- General functionalities(Score, Gameover, Restart)

Description :

1. UI & map development (Core) –

Members Involved :

- BHINDI MEET GHANSHYAMBHAI – 202501122
- PIYUS PRATIK JENA – 202501117
- RAVI KARIYA – 202501112

UI & Map was one the core part of this project as the game is mainly based on its appearance and interactions with Users.

Contributions :

1. BHINDI MEET GHANSHYAMBHAI – 202501122

I have majorly contributed in the integration work, I have made the UI of file management(login/signup), along with that I have mainly designed the Map array so that it matches the Map of real game. I integrated the movement logic code of both PacMan and Ghost, Music, Login & SignUp pages, Pause, Settings, Menu,etc.

2. PIYUSH PRATIK JENA – 202501117

I have contributed to the development of the UI of GameOver, LeaderBoard and Settings, that includes the volume control, the reset settings when Game is Over and updating the LeaderBoard, getting Sorted data from file and displaying it in Leaderboard.

3. RAVI KARIYA – 202501112

I have mainly contributed in the development of the map, that is its logic development and design to some extent. Also I have been a part in the overall Integration of all the functionalities to the main code.

Code Explanations :

The code flow starts with loading the Header files, then defining some global variables like Screen Width/Height,etc.

Then the Map of the Game is defined.

The Logic of Map is given below :

The is a 2D array of 32 by 28 tiles, each tile of length 22 pixels

– 0 represents blank tile (nothing to be drawn in it)

– 1 represents tile with palette(drawing a small circle in the middle representing palette)

– 9 and 9 series(90 to 94) represents double line wall

– 8 and 8 series(80 to 84) represents single line wall

– 51 to 56 are special cases of double line walls

– 7 and 7 series(70 to 74) represents walls of ghost cage

– 6 represents Gateway

The Original Map “OGMap” is copied by using memcpy() function into another 2D array Map so that the Original Map is not lost while the Game updates the second Map.

An enum “State” is used for getting different States of the Game.

Then by using the default Raylib function InitWindow() and InitAudioDevice() to initialize Window and Speaker for upcoming uses.

The SetTargetFPS() function ensures VSync.

The while is the **Main Game Loop** :

Inside the loop, GameState is passed in switch, which has all different cases according to different game parts.

- Case Login:

  This is the default case which opens first when you run the code.
  Its part is contained in the LoadLoginPage() function.

  LoadLoginPage() function details :

  ‘myFont’ and ‘textFont’ are two custom font styles passed as arguments.

  The Vector2 mousePos variable has the position of the mouse.

  Different variables and functions of Raylib are used to print all the Text and Rectangles, it takes Input from User and then the File Handling logic checks if the credentials entered are correct or not.

  If the User is not registered, by clicking on SignUp, the GameState changes to SignUp.

  If the Login Credentials are correct, then the GameState changes to Menu.

- Case SignUp:

  The SignUp page is similar to the Login Page, the only difference is that in the SignUp page a new Username and Password will be created in the Users file.

  After Signup, the User will have to go back to the Login Page again for Login.

- Case Menu:

  The menu mainly acts as a navigation point to all the other States.

  It changes the Gamestate according to the button pressed.

- Case LeaderBoard:

  LeaderBoard prints the HighScores of top ten Users present in the File.

- Case SettingsM:

  SettingsM means the Settings is called from the Menu.

  The Settings State contains a page that can change the volume of the Overall Game.

  The Volume Settings is loaded by the VolumeSettings() function which also takes the previous state as an argument.

  This will display a slider with a knob which controls the Overall Volume.

  Since Settings can be accessed from both Menu and Pause, this helps us use only one function for both the Cases.

- Case SettingsP:

  SettingsP means the Settings is called from the Pause State.

  It works exactly the same as the previous one.

- Case GameOver:

  This case calls the Quit() function, it checks if the current score is greater than the previous high score, if it is greater then it updates the new high score in User data.

  When we press the Menu button, it resets all the other things to its initial values using the reset() function.

- Case Pause:

  Whenever the user presses 'Escape Key' while playing the game, all the game inputs and movement are stopped, and another Menu similar to the “Menu” appears, which contains Resume, Settings and Quit buttons.

- Case Playing

  This is the Main game State, it contains all the Game functions that are required for the Game to run properly.

– RenderMap()

This function renders the entire map.

– HandleSound()

This functions handles all the sound

The introduction sound is checked by a variable ‘introsound’ so it runs only once

The background runs in loop continuously.

– RenderPac()

It renders the Pacman character, creating its mouth ‘s opening and closing animation by using a Sin function to continuously change the Sector.

– RenderGhost()

It renders the Ghost, create its wiggle effect of legs by using Sin function to continuously change the length of its leg

Also it has given a blinking effect, it blinks randomly in intervals of 2 to 5 seconds.

Rest all functions are handling movement of pacman and ghost and also for detecting their collision.

2. Movement of Ghost and Pac-Man (Core) –

Members Involved :

- DESAI SANVI KALPIT – 202501113
- ARYAN GAMBHAVA – 202501116
- PADSUMBIYA DEV MAHESHBHAI – 202501121

The **Movement of Pac-Man** is controlled directly by the player's input (up, down, left, right) but is limited by the maze **walls**. The **Ghost Movement** is implemented using BFS(Breadth-First Search) algorithm.

Contributions :

1. DESAI SANVI KALPIT – 202501113

My contribution in the code is writing code for pacman's movement and ghost movement. Function used in pacman movement is pacmanupdate() and functions used in ghost movement are GetPacmanTargetTile() and GhostUpdate().

2. ARYAN GAMBHAVA – 202501116

I have contributed in designing the logic for ghost AI and pacman movement using AI tools. The ghost AI uses BFS (Breadth First Search) Algorithm to find the shortest possible path to pacman. Also the movement of ghost and pacman are only executed when they are validated by several functions so that both moves only in walkable paths and not through walls.

3. PADSUMBIYA DEV MAHESHBHAI – 202501121

I have contributed to the core part of the Pac-Man movement and ghost movement. The central chasing part of the ghost to Pac-Man was accomplished by the Breadth-First Search Pathfinding process, with the help of ChatGPT to approach the shortest path to Pac-Man. Also, make sure that the movement of Pac-Man and ghost is only in the allowed path, not through the walls.

Code Explanation:

PacMan Movement:

– Function used: pacmanupdate(), checkwall()

1. Reads user input to set pacman direction. Input can be done in two ways: by pressing arrow keys and also by letter keys.
2. After getting the input, the pacman's direction is updated using enum values.
3. It is implemented using if else statements.
4. It detects the walls by using the function checkwall().
5. The function is boolean type. It returns true if there is a wall and false if there is no wall. The wall is checked if it is within the map borders.
6. It eats the pellet after coming to the next tile so map now becomes 0 and it is blank space now.
7. The score increases by 10 whenever it eats a pellet. The score is calculated by the function UpdateScore().

Ghost Movement:

– Function used: GetPacmanTargetTile(),ghostupdate()

- 1. GetPacmanTargetTile()

  Use: To find and set the target tile for the ghost.

- 2. ghostupdate()

  Use: To update the position of ghost to chase pacman.

- **1. GetPacmanTargetTile():**
- First it gets pacman's position and converts it into tile.
- It takes Pacman's current tile as its first tile.
- Gets the pacman's current tile and sets it as target and stores it in TargetX and TargetY.

- **2. ghostupdate():**
- First of all it gets the ghost's position in pixels and converts into Tile and stores in ghostTileX and ghostTileY.
- Now, it gets the target tile from the function GetPacmanTargetTile().
- Here, BFS (Breadth First Search) algorithm is used using queues to chase the pacman using the shortest possible path.

- Initializes BFS and it runs until the pacman's tile is found or the queue becomes empty.
- visited[][] grid is defined to check if ghost has visited that tile to check pacman's position. parent[][] grid is defined for restoring the shortest possible path.
- If the target tile is in range, movable and not visited then adds the neighbour tile to the queue and sets it in the parent[][] grid.
- If it is found then it backtracks it until the ghost tile and sets the next tile for the ghost in the parent[][] grid.
- The tiles are constantly added and removed from the queue as the ghost moves.
- Current tile of ghost is removed from the queue using pop function.
- Tile from parent[][] grid is added to the queue using push function to which the ghost moves next.
- If current tile of ghost and target tile coincides then the loop breaks immediately.
- Initially found is defined to be 0. If found==1 i.e. if it finds pacman's tile in range, it starts from the target tile and calculates ghost's next tile.
- Converts tile into pixel and stores in tx and ty which is the next tile to which the ghost moves.
- Moves ghost one speed toward that pixel.
- This continues everytime until the ghost captures the pacman and game ends.

3. File/User data management (Main functionality)–

Members Involved :

- ☐ PRANAV MADHAV BIRAJDAR – 202501115
- ☐ GOHIL ROHAN DIPAKBHAI – 202501120

The project will use **file management** to permanently **store user data** like high scores and usernames on the computer. This ensures that player rankings and the top scores achieved are remembered and displayed every time.

Contributions :

1. PRANAV MADHAV BIRAJDAR – 202501115

I mainly focused on designing the logic and writing the core parts of file handling.

2. GOHIL ROHAN DIPAKBHAI – 202501120

I contributed to debugging and testing. We fixed issues together and combined our contributions to successfully complete the project.

Flow Chart :

1) Main function calls “LoadUsers()” function which reads required text file and saves data in users[] array.

2) When the user selects the sign up button and submits username password, FindUser() function checks if account already exists or not. If it exists then it prints account already exists, if it doesn’t then the SaveNewUser() function saves the credentials.

3) When a user selects the login button and enters credentials , check login() checks if the credentials are correct.If login is successful , CheckLogin() returns the index of that user inside the users[] array, otherwise it shows error.

4) Now, GameOver() function returns the score after the match is complete, if it is higher than users.highscore , then it is updated.After updating , sortUsers() function is called which uses bubblesort to sort accounts based on highscores and it also saves the new list in the text file.

5) If the user selects Leaderboard then LoadLeaderBoard() displays the leaderboard.

Code Explanations :

1) FindUser()

It takes string as input,it uses for loop, in which the entered username is compared with the existing usernames in text files. It returns the index of the account if the usernames are same, else it returns -1.

```
int FindUser(const char *username)
{
    for (int i = 0; i < userCount; i++)
    {
        if (strcmp(users[i].username, username) == 0)
        {
            return i; // Found at index i
        }
    }
    return -1; // Not found
}
```

2) CheckLogin()

It takes two strings as input which are username and password entered by the user. It also goes through both username and password existing in text file and if both match then it returns index of the account, otherwise it

returns -1.

```
int CheckLogin(const char *username, const char *password)
{
    for (int i = 0; i < userCount; i++)
    {
        if (strcmp(users[i].username, username) == 0 &&
            strcmp(users[i].password, password) == 0)
        {
            return i; // Found, return user index
        }
    }
    return -1; // Invalid login
}
```

3) LoadUsers()

If the file exists, it loads from it, if it doesn't exists then file pointer "file" gets value Null which is returned by fopen function. A while loop is used to read data from the text file which stops if the end of file is reached or userCount reaches MAX_USERS.

```c
void LoadUsers()
{
    FILE *file = fopen(USERS_FILE, "r");
    if (file != NULL)
    {
        // Read users until EOF or array is full
        while (userCount < MAX_USERS && fscanf(file, "%s %s %d",
                                                users[userCount].username,
                                                users[userCount].password,
                                                &users[userCount].score) == 3)
        {
            userCount++;
        }
        fclose(file);
    }
}
```

4) SaveNewUser()

If the file pointer doesn't receives Null value from fopen function, then users.txt is opened in append mode. fprintf is used to store data and then the file is closed.

```c
void SaveNewUser(User user)
{
    FILE *file = fopen(USERS_FILE, "a");
    if (file != NULL)
    {
        fprintf(file, "%s %s %d\n", user.username, user.password, user.score);
        fclose(file);
    }
}
```

5) SortUsers()

It uses bubblesort to sort accounts based on their highscore and adjusts the loggedInUserIndex and after sorting it rewrites the file.

```
void SortUsers()
{
    for (int i = 0; i < userCount - 1; i++)
    {
        for (int j = 0; j < userCount - i - 1; j++)
        {
            if (users[j].score < users[j + 1].score)
            {
                User tmp = users[j];
                users[j] = users[j + 1];
                users[j + 1] = tmp;

                if(loggedInUserIndex == j){
                    loggedInUserIndex++;
                }
                if(loggedInUserIndex == j+1){
                    loggedInUserIndex--;
                }
            }
        }
    }

    FILE *file = fopen(USERS_FILE, "w");
    if (file != NULL)
    {
        for (int i = 0; i < userCount; i++)
        {
            fprintf(file, "%s %s %d\n",
                    users[i].username,
                    users[i].password,
                    users[i].score);
        }
        fclose(file);
    }
}
```

4. Music –

Members Involved :

- KATHAN JITENDRAKUMAR SAMDANI – 202501118

The game will feature **music and sound effects** to enhance the experience. This includes playing a looping **background theme** while the player is active and incorporating short **sound clips** for actions like eating pellets or losing a life.

Contributions :

I have contributed in making the Sound effects, there are 4 different sounds playing for different conditions.

5. General functionalities –

Members Involved :

- VADHVANA VISHESH JIGNESHKUMAR – 202501114
- RAVAL DHARMIK PRAVINBHAI – 202501119

The project includes several general functions to manage the game, such as tracking the **Score**. It handles the **Game Over** state and allows players to **restart** a new game.